

Textile Manufacturing Operations Management System

Enterprise PostgreSQL Database Architecture & Technical Whitepaper

Author: Raj Kachhadiya | Tech Stack: PostgreSQL, ANSI SQL | Standard: Boyce-Codd Normal Form (BCNF)

1. Executive Summary & System Overview

The **Textile Manufacturing Operations Management System** is a production-grade, multi-facility relational database designed to manage end-to-end operational workflows for large-scale textile manufacturing enterprises. The system coordinates complex supply chains, multi-step production pipelines (Spinning, Weaving, Dyeing, Packing), raw material procurement, multi-warehouse stock allocations, quality control (QC) batch testing, worker shift wage aggregations, and bulk order fulfillment.

The database is implemented in **PostgreSQL** and comprises **21 logical entities** synthesized into **34 physical relations**. Every relation in the schema has been formally derived from scratch and verified to satisfy **Boyce-Codd Normal Form (BCNF)**, guaranteeing complete elimination of data redundancy, update anomalies, and join anomalies while preserving critical business constraints.

2. Domain Architecture (7 Operational Modules)

- 1. Procurement & Supplier Management:** Manages raw material purchases, unit price negotiations, supplier contacts, multi-location vendor listings, and transport logistic partners.
- 2. Departmental Workforce & HR Allocation:** Manages employees, department managers, supervisors, shift schedules, attendance, dependents, bank/insurance details, and worker wage aggregations.
- 3. Multi-Phase Production Execution:** Tracks step-by-step production workflows (Spinning, Weaving, Dyeing & Finishing, Packing), sequence executions, timestamps, phase resource consumption, and yield wastage.
- 4. Quality Control (QC) & Batch Inspection:** Monitors inspection events, batch packaging, fabric roll grading, storage rack allocations, and pass/fail batch quality statuses.
- 5. Multi-Warehouse Inventory Control:** Tracks warehouse capacities, zone allocations, raw material stock levels, reorder thresholds, and consignment delivery receipts.
- 6. Invoicing & Financial History:** Integrates purchase order aggregations with invoice tracking, tax amounts, discounts, due dates, and payment statuses (pending, partial, paid).
- 7. Logistics & Consignment Tracking:** Tracks transport vehicles, driver licenses, consignment delivery timestamps, and expected arrival schedules.

3. Entity-Relationship & Schema Architecture (34 Tables)

The schema design utilizes robust referential integrity rules (FOREIGN KEY ... ON DELETE CASCADE), domain validation constraints (CHECK), non-null constraints, and UNIQUE key declarations. Below is the complete physical table catalog:

Table Name	Primary Key	Key Constraints	Domain Purpose
Department	Dept_ID (PK)	Dept_Name (UNIQUE), Manager_Emp_ID (FK)	Department organizational units and manager assignments.
Employee	Emp_ID (PK)	Company_Email (UNIQUE), Dept_ID (FK), Supervisor_ID (FK)	Staff profiles, salaries, job titles, and supervisory hierarchy.
Company	Company_ID (PK)	Company_Email (UNIQUE), GSTIN (UNIQUE), Rep_Emp_ID (FK)	Vendor, supplier, customer, and transporter company directory.

Table Name	Primary Key	Key Constraints	Domain Purpose
Company_Type	(Company_ID, Company_Type)	Company_ID (FK)	Decomposed multi-valued company classification categories.
Company_Location	(Company_ID, PIN)	Company_ID (FK), City, State	Multi-location address entries per company.
Shift	Shift_ID (PK)	Shift_Name, Start_TS, End_TS	Operating shift definitions and working hour windows.
OrderInvoice	Invoice_ID (PK)	Invoice_TS, Total_Amount, Tax, Discount, Status, Due_Date	Financial billing records and payment status tracking.
RawMaterial	Material_ID (PK)	Material_Name, Category, Unit_Of_Measure, Reorder_Level	Raw material master inventory catalog.
Warehouse	Warehouse_ID (PK)	Zone, Capacity, Dept_ID (FK)	Physical storage facilities and capacity parameters.
Vehicle	VehicleNo (PK)	License_Number (UNIQUE), Company_ID (FK)	Logistics transport vehicle and driver details.
PurchaseOrder	Order_ID (PK)	Order_TS, Status, Dept_ID (FK)	Procurement purchase order headers.
Consignment	Consignment_ID (PK)	VehicleNo (FK), Warehouse_ID (FK), Order_ID (FK)	Shipment delivery tracking and arrival timestamps.
ExtraEmployeeDetails	Emp_ID (PK, FK)	Bank Details, Insurance Details, Address, PIN, City, State	1:1 extension table storing payroll bank, policy, and address info.
Dependent	(Emp_ID, Dep_Name)	Emp_ID (FK), Relationship, Date_Of_Birth	Employee family dependents (weak entity).
Attendance	(Emp_ID, Att_Date)	Emp_ID (FK), Status, Check_In_TS, Check_Out_TS	Daily employee attendance logs (weak entity).
Employee_Shift	(Emp_ID, Shift_ID)	Emp_ID (FK), Shift_ID (FK)	Employee shift assignment junction table.
FinalMaterial	Product_ID (PK)	Product_Name, Grade, Fabric_Type, Colour, Reorder_Level	Finished fabric product catalog.
ProductionOrder	Order_ID (PK)	Dept_ID (FK), Product_ID (FK), Expected_Qty_Output	Production manufacturing order headers.
PhaseExecution	(Order_ID, Seq_No)	Dept_ID (FK), Product_ID (FK), Qty_Input, Qty_Produced	Multi-step production phase step execution records.
PhaseTimeLog	(Log_ID, Order_ID, Seq_No)	Order_ID/Seq_No (FK), Event, Event_TS, Reason	Operational event and downtime logs per execution phase.
PhaseResourceConsumption	(Consumption_ID, Order_ID, Seq_No)	Order_ID/Seq_No (FK), Recorded_TS	Phase resource consumption header records.
ResourceConsumed	(Consumption_ID, Order_ID, Seq_No, Resource_Type)	Consumption Header (FK), Unit, Qty_Consumed	Multi-valued resource items consumed per execution phase.
Worker	Worker_ID (PK)	Worker_Name, Phone, Role, Emp_ID (FK)	Operational plant worker profiles owned by Employees.
PackingBatch	Packing_ID (PK)	Dept_ID (FK), Prod_Order_ID/Phase_Seq_No (FK)	Quality inspection and batch packaging headers.
FabricRoll	Roll_ID (PK)	Packing_ID (FK), Warehouse_ID (FK), Product_ID (FK)	Individual packaged fabric rolls and rack storage locations.
Company_RawMaterial	(Company_ID, Material_ID)	Company_ID (FK), Material_ID (FK)	Supplier to raw material capability junction table.
RawMaterial_Warehouse	(Material_ID, Warehouse_ID)	Qty_Stock	Raw material stock quantities per warehouse facility.

Table Name	Primary Key	Key Constraints	Domain Purpose
RawMaterial_Department	(Material_ID, Dept_ID)	Material_ID (FK), Dept_ID (FK)	Raw material allocation junction per department.
Consignment_RawMaterial	(Consignment_ID, Material_ID)	Qty_Delivered	Delivered raw material line items per consignment shipment.
PurchaseOrder_RawMaterial	(Order_ID, Material_ID)	Unit_Price	Negotiated unit price line items per purchase order.
PO_Company	(Order_ID, Company_ID)	Expected_Order_Completion, Invoice_ID (FK, UNIQUE)	Purchase order to vendor aggregation linked 1:1 with OrderInvoice.
ProductionOrder_RawMaterial_Warehouse	(Order_ID, Material_ID, Warehouse_ID)	Qty_Required	Ternary relationship quantifying raw material requirements.
Worker_Department	(Worker_ID, Dept_ID)	Worker_ID (FK), Dept_ID (FK)	Worker department assignment base junction.
WorkerDept_Shift	(Worker_ID, Dept_ID, Shift_ID)	Wages	Ternary aggregation tracking worker wages per department & shift.
Worker_PackingBatch	(Worker_ID, Packing_ID)	Worker_ID (FK), Packing_ID (FK)	Worker packing batch assignment base junction.
WorkerPacking_Shift	(Worker_ID, Packing_ID, Shift_ID)	Wages	Ternary aggregation tracking worker wages per packing batch & shift.
Worker_Warehouse	(Worker_ID, Warehouse_ID)	Worker_ID (FK), Warehouse_ID (FK)	Worker warehouse assignment base junction.
WorkerWarehouse_Shift	(Worker_ID, Warehouse_ID, Shift_ID)	Wages	Ternary aggregation tracking worker wages per warehouse & shift.

4. Formal Normalization Analysis (BCNF Proof)

Boyce-Codd Normal Form (BCNF) Standard: A relation *R* is in BCNF if and only if for every non-trivial Functional Dependency $X \rightarrow Y$ in *R*, the determinant *X* is a superkey (candidate key) of *R*.

Verification Results: Every functional dependency derived from the physical schema was evaluated. Because all single-entity tables have candidate keys determining all non-key attributes, all binary junction tables have composite keys as their sole determinants, and all ternary aggregation tables (`WorkerDept\_Shift`, `ProductionOrder\_RawMaterial\_Warehouse`) have composite candidate keys determining functional attributes (such as `Wages` or `Qty\_Required`), **all 34 relations satisfy BCNF.**

5. Key Business Queries & Analytical Use Cases

The repository contains over 35 analytical SQL queries covering essential business decision-making scenarios. Below are sample queries demonstrating query optimization, CTEs, and aggregation analytics:

Query Sample 1: Supplier Expenditure & Invoicing Summary:

```
SELECT  c.Company_Name, ct.Company_Type, COUNT(oi.Invoice_ID) AS Invoice_Count,
        SUM(oi.Final_Amount) AS Total_Invoiced, SUM(oi.Tax_Amount) AS Total_Tax_Paid
FROM    Company c
JOIN    Company_Type ct ON ct.Company_ID = c.Company_ID
JOIN    PO_Company poc  ON poc.Company_ID = c.Company_ID
JOIN    OrderInvoice oi ON oi.Invoice_ID = poc.Invoice_ID
GROUP BY c.Company_Name, ct.Company_Type
ORDER BY Total_Invoiced DESC;
```

Query Sample 2: Production Phase Yield & Output Variance Analysis:

```
SELECT  po.Order_ID, fm.Product_Name, po.Expected_Qty_Output, pe.Qty_Produced,
        (pe.Qty_Produced - po.Expected_Qty_Output) AS Variance, pe.Status
FROM      ProductionOrder po
JOIN      FinalMaterial fm ON fm.Product_ID = po.Product_ID
JOIN      PhaseExecution pe ON pe.Order_ID = po.Order_ID
WHERE     pe.Qty_Produced IS NOT NULL
ORDER BY po.Order_ID;
```

6. Technical Stack & Deployment Guide

Database Engine: PostgreSQL 12+

Execution Order:

1. sql/schema.sql — Creates table definitions, constraints, and foreign keys.
2. sql/seed_data.sql — Populates comprehensive seed data across all 34 tables.
3. sql/queries.sql — Executes analytical query scenarios.

Secure Connection Setup (Environment Variables):

```
export PGHOST=${PGHOST:-localhost}
export PGPORT=${PGPORT:-5432}
export PGDATABASE=${PGDATABASE:-textile_db}
export PGUSER=${PGUSER:-postgres}
psql -h $PGHOST -p $PGPORT -U $PGUSER -d $PGDATABASE -f sql/schema.sql
```